模型评估

Module 3 · Topic 3 — 知识点完整整理

ARTI2001 Machine Learning · 2026 Semester 2

涵盖 **Topic 3 三课**

训练/验证/测试的角色 · **数据泄漏** · 欠拟合与过拟合的证据

学习率证据 · 重复训练证据 · **基于验证集的早停**

混淆矩阵 · **accuracy / precision / recall / F1**

ROC / AUC 与阈值权衡

M3 测验：Week 13（2026-11-09 ~ 11-15）· 110 分钟 · 开卷 · 20 选择×3 + 4 大题×10

目录

M3 T3 第 1 课 · 数据划分与训练证据

· 0. 主旨
· 1. ★★ 三种数据角色
· 2. ★★ 先决定「新数据」是什么意思
· 3. ★★★ 信息泄漏（本课最容易出大题的点）
· 4. ★★ 完整的一轮开发流程（practical 用的 refit 工作流）
· 5. 交叉验证（一次划分不稳时用）
· 6. ★★ 诊断表（措辞非常审慎，要照着答）
· 7. ★★ 学习率的一步实验
· 8. ★★★ 重复更新：一个很值得注意的现象
· 9. 总结自测
· ★ 速查卡

M3 T3 第 2 课 · 混淆矩阵与四个指标

· 0. 第一件事：先定义哪一类是「正类」
· 1. ⚠⚠⚠ 混淆矩阵的摆法——本课的约定（极易混，必记）
· 2. 四个核心指标
· 3. ★★ 看分母来选指标
· 4. ★ 官方算例（12 个案例）
· 5. ★★ 阈值与指标的关系
· 6. ★★★ 从指标到决策：用代价把阈值定下来
· 7. 配套活动：Metrics by hand（20 位顾客，广告点击）
· 8. 总结自测
· ★ 速查卡

M3 T3 第 3 课 · ROC 曲线与 AUC

· 0. 主旨
· 1. 先把定义摆清楚
· 2. ★★ 怎么构造这条曲线（五步）
· 3. ★★ 官方算例：四个样本
· 4. ★★ 梯形法算 AUC
· 5. ★★★ 用「正负配对」复核面积（更好用的办法）
· 6. ⚠ AUC 数值的解读分寸（措辞很讲究）
· 7. ★★★ AUC 不能决定什么（四条，最可能出选择题）
· 8. ★★ 同一组样本的 PR 视角（对照着看）
· 9. ★★★ 稀有正例的那个算例（本页最该记的数字）
· 10. 总结自测
· ★ 速查卡

M3 T3 第 1 课 • 数据划分与训练证据

源页面：Dataset splitting and training evidence ｜ 整理日期：2026-10-14 ｜ 对应考纲：Topic 3「训练/验证/测试的角色、泄漏与模型选择；欠拟合过拟合的证据； ｜ 学习率证据与重复训练证据；基于验证的早停」—— **这一页几乎覆盖了 T3 的前半个考纲**

0. 主旨

> **设计一个「像真正的新案例」的评估。**
>
> 先想清楚预测场景，再分配不同的数据角色，
>
> **然后在预处理和模型选择的全过程中守住这些边界。**

1. ★★ 三种数据角色

角色	用来做什么
训练集	拟合权重**以及其它学到的量**——⚠ **包括缩放参数和缺失值填补参数**
验证集	比较各种**选择**：隐藏层大小、激活函数、正则化、**停止点**、**判定阈值**。这些都叫**开发决策**
测试集	评估**已经选定并冻结**的流程。⚠⚠ **不得用它的标签或结果来做这个选择**

> ⚠ **70/15/15、80/10/10 这类比例是举例，不是规则。**
>
> 要留出足够多的相关案例——**包括足够多的正例事件。**
>
> 具体数量取决于**数据集大小、记录之间的依赖性、以及预期的预测场景。**

⚠ 划分能给什么、不能给什么

> **当留出样本能代表预期的部署场景、且避免了信息泄漏时**，一个设计良好的划分**给出的是对未见数据表现的估计。**

> ⚠⚠ **它不保证泛化，也不保证在分布漂移下的稳健性。**

没有留出集会怎样

- **过拟合**：灵活的模型可能学到训练集特有的模式或噪声。

⚠ 页面措辞很审慎：**这是一个需要诊断的风险，不是训练的必然后果。**

- **评估受限**：只看训练表现**无法可靠估计未见样本的表现，也看不出泛化差距。**

⚠ **训练表现差本身就已经能提示欠拟合或优化问题**；**把训练和验证结果对比，才是更强的诊断证据。**

2. ★★ 先决定「新数据」是什么意思

划分方式	什么时候用
分层划分 Stratified	分类类别不平衡时，**保持各类比例**
分组划分 Grouped	**把同一个人/病人/设备/站点的记录放进同一个子集**
按时间划分 Time-ordered	预测未来时，**用较早的观测训练、较晚的观测验证**

> ⚠⚠ **随机划分不是自动就合适的。**
> **评估设计应当复现「真正没见过的数据将以什么方式到来」。**

★ Study check：一个病人贡献 10 条记录，把其中一些分到训练、另一些分到测试，为什么会得到误导性的表现?

> 因为模型可以**记住这个病人特有的特征**（而不是学到可推广的规律）。
>
> 测试集里那几条记录来自**模型已经见过的同一个人**，所以测试分数**偏乐观**——
>
> **它衡量的不是「对新病人的表现」。**
>
> ⇒ **这种情况必须用分组划分。**

3. ★★★ 信息泄漏（本课最容易出大题的点）

> **数据泄漏**
>
> = **在预期预测场景中拿不到的信息，影响了训练或选择。**
>
> 途径：**标签、学到的预处理、重复记录、未来信息。**

★ 那个极小的缩放算例（很能说明问题）

> 训练值 $[0, 2]$，均值 $= 1$
> 留出值 100 应该这样中心化： $100 - 1 = 99$ ✓ 正确
>
> 若把三个值放在一起算均值： $(0+2+100)/3 \approx 34$
> ⇒ 留出的那个案例「改变了连训练数据本身的表示方式」

> ⚠⚠ **页面的关键一句：从输入里学习、即使没用到标签，一样可能越过评估边界。**

★★ 正确的顺序（五步，要能背）

1. 先把数据划分成 训练 / 验证 / 测试
2. 只用训练集计算填补参数和缩放参数
3. 把这些学到的参数应用到验证集和测试集 —— 不要重新拟合
4. 用验证结果选择模型和超参数
5. 所有选择固定之后，在没碰过的测试集上评估一次

k 折交叉验证里怎么办

在每一个训练折内部单独拟合预处理，再应用到那一折的验证部分。

软件的 pipeline 有助于强制这条边界。

⚠⚠ **在交叉验证之前就对整个数据集做预处理，等于把验证信息泄漏进了训练。**

⚠ 自测：缩放器不用标签，是不是可以在交叉验证之前对全部开发数据拟合一次?

不可以。

见上面那个 [0,2] vs 100 的例子——**不用标签也会泄漏。**

★★ 和 PCA 的联系（M2 学过的东西在这里回收）

PCA 和缩放都是从输入里学习的，即使不用标签。

如果它们用在有监督模型之前，**必须在每个训练划分或折的内部拟合。**

⚠⚠ **「无监督」不等于「可以安全地在测试数据上拟合」。**

4. ★★ 完整的一轮开发流程（practical 用的 refit 工作流）

1000 个独立样本 → 700 训练 / 150 验证 / 150 测试

① 只用那 700 个拟合缩放器和每个候选模型
② 在 150 个验证样本上比较候选，用为任务选定的指标
③ 需要的话，用验证集的预测来选阈值
④ 选择固定后，可以把选定的流程「重新拟合一份新的副本」
在合并的 850 个（训练＋验证）样本上 —— 它的缩放器在这 850 个上重新拟合
⑤ 在 150 个测试样本上评估这个冻结的流程

⚠⚠ **不要在报告那个测试结果之前，用全部 1000 个重新拟合。**

⚠ 两条附带的警告

1. **在 refit 之前选定的阈值，可能迁移得不完美**——因为 **refit 之后分数会变。**

应当**用开发数据或 out-of-fold 预测检查稳定性**；

⚠⚠ **绝对不要用最终测试集的标签去「修正」阈值。**

1. ⚠⚠ **如果结果促使你继续开发，那个旧的测试集就已经成了开发历史的一部分**——

要做出新的最终结论，需要一次全新的独立评估。

5. 交叉验证（一次划分不稳时用）

- **k** = 折数。**每一轮在 k−1 折上拟合一个全新的模型**，在剩下那一折上评估。
- **超参数**是「模型大小、正则化强度」这类**选择**，**用验证证据选出来，而不是像权重那样直接拟合出来。**
- **数据稀缺时**，单次训练-验证划分的估计**不稳定**，因为结果强烈依赖于哪些观测进了验证集。
- ⚠ **每一轮的预处理都要在该轮的训练部分重新拟合。**
- **可行时仍要留出一个独立的、有代表性的测试集**做最终评估。

（若全部数据都必须用于开发，**嵌套交叉验证**可以把调参和评估分开；细节本课可选。）

★ 例子与一条重要限定

> 五折的验证准确率 **0.80, 0.82, 0.78, 0.84, 0.81 ⇒ 平均 0.81。**

> ⚠⚠ **这不是五次独立的测试实验，因为它们的训练集互相重叠。**
>
> **这个平均值仍然依赖于抽样和划分的假设，并且带有不确定性。**

> **每个候选配置都要有它自己的一套全新的交叉验证拟合。**
>
> 选定之后，在允许使用的开发数据上拟合一个全新的最终模型。

⚠ 自测：验证表现很差，该不该去看测试结果来挑一个更好的模型？

> **不该。**
>
> 一旦用测试结果做选择，**它就不再是独立的最终评估了。**

6. ★★ 诊断表（措辞非常审慎，要照着答）

模式	典型证据	解读
可能欠拟合或优化困难	**训练和验证都差**	模型可能**太受限、特征不足**、或**优化失败**。⚠ **这个模式本身并不能断定「统计偏差高」**
可能过拟合或划分不匹配	**训练很强，但验证明显更差**	可能过拟合。⚠⚠ **在把差距归因于方差之前，还要检查分布差异、验证样本过小、以及泄漏**
验证表现令人鼓舞	**训练和验证都强且相当接近**	学到的模式**迁移到了未见的验证样本**

> ⚠⚠ **这些是诊断模式，不是证明。**
> 用验证数据比较模型、调超参数；**在最终模型和决策流程选定之前，测试集一直不要碰。**

7. ★★ 学习率的一步实验

⚠⚠ **下表的每一行都是「从同一个初始参数重置后，只走一步」的独立实验，不是一次运行的连续步骤。**

数据：四个观测 ([1,2],y=1)、([−1,−2],y=1)、([2,−1],y=0)、([−2,1],y=0)

起始平均 BCE = **0.580352**

学习率 η	一步之后的平均 BCE
0	**0.580352**（参数没变）
0.1	0.574996
1	0.533347
10	**0.371611** ← 这一步最低
100	**9.965953** ← ⚠ 飞了

> ★ **小步子能改善损失；非常大的步子会过冲，或者改变「哪些 ReLU 单元是激活的」。**

> ⚠⚠ **这些结果只属于这个批次和这个起点，不能据此确定一个普适的最佳学习率。**

⚠ 自测：η=10 在这张表里一步后损失最低，它就是后续每一步、每个数据集的最佳学习率吗？

> **不是。**
> 这只是**单步**的结果。**一步走得远 ≠ 长期收敛得好**——大学习率后续可能震荡或发散。

8. ★★★ 重复更新：一个很值得注意的现象

⚠ **这张表是一次连续的训练运行**（同样四个观测、η=0.1、全批量、每次更新前都重新做前向/反向）。

本例每个 epoch 做一次全批量更新。

完成的更新数	平均训练 BCE	p：[1,2] y=1	p：[−1,−2] y=1
0	0.580352	0.6615	**0.4750**
1	0.574996	0.6664	0.4749
5	0.552753	0.6867	0.4748
10	0.527730	0.7158	0.4750
20	0.484156	0.7770	0.4759
50	0.406138	0.8987	0.4788
100	**0.369420**	**0.9592**	**0.4859** ← 仍在 0.5 以下

★★ 这张表最该带走的一条

> **平均损失在降，但两个正例的改善完全不对等。**
>
> **跑完 100 次更新，第二个正例的概率仍然低于 0.5 判定阈值**（0.4859）。

为什么?

> **它的两个隐藏 ReLU 激活从一开始就是零**
>
> ，
>
> ⇒ **它的正类分数只依赖于输出偏置**，隐藏层那条路对它完全没用。

> ⚠⚠ **平均损失更低，不代表每个样本都被正确分类。**

关于收敛与停止

- ⚠ **一次损失下降不能确立收敛。** 要看重复更新下的行为，**在一个明确的停止准则下**考察目标函数或参数的变化。
- ⚠⚠ **即便看起来稳定了，也可能只是优化变慢或卡住了，而不是到了最好的解。**
- **这个演示是在固定的 100 次更新预算后停的，不是因为证明了收敛。**
- ★ **真实任务里应该**：**定期监控验证表现、用开发数据定一条停止规则**（例如「连续几次检查都没有改善就停」）、**保留验证表现最好的那个检查点。测试数据留给最终评估。**

⚠ 自测：100 次更新后第二个正例概率约 0.4859，阈值 0.5 下预测成哪一类？平均损失为什么还能更低？

> **预测成类别 0 —— 判错了**
>
> （0.4859 < 0.5）。
>
> **平均损失仍能下降，是因为其它样本改善得足够多，把平均拉下来了。**

9. 总结自测

Q：某模型训练准确率 98%、验证准确率 72%，能得出什么结论？下一步该做什么？

> **这是「训练很强但验证明显更差」的模式 ⇒ 可能过拟合。**

⚠ 但按第 6 节的要求，**在归因于方差之前还要检查**：

分布差异、验证样本是不是太小、以及有没有数据泄漏。

下一步（都在验证集上做，不碰测试集）：

加正则化 / 减小模型 / 增加数据 / 早停；并**复核划分方式是否合适**（该不该用分组或时间划分）。

★ 速查卡

三角色： 训练=拟合(含缩放与填补参数) 验证=选择(含阈值与停止点) 测试=只评估冻结流程一次

泄漏五步法： 先划分 → 只在训练集上算预处理参数 → 原样应用到验证/测试
→ 用验证选模型 → 全部定好后在测试集上评估一次
⚠ 不用标签也会泄漏（[0,2] 均值 1；混入 100 后变 34）
⚠ 「无监督」≠「可以在测试集上拟合」（PCA 同理）

划分方式： 不平衡→分层 同源记录→分组 预测未来→按时间
⚠ 随机划分不是自动合适的

refit 流程： 700训/150验/150测 → 选定后可在 850 上重拟合 → 在 150 测试上评一次
⚠ 不要用全部 1000 重拟合后再报测试结果
⚠ 测试集一旦参与选择，就需要一次全新的独立评估

诊断： 两边都差→可能欠拟合/优化困难（⚠ 不等于高偏差）
训练强验证差→可能过拟合（⚠ 先排除分布差异、验证集过小、泄漏）
⚠ 都是「诊断模式」，不是证明

学习率： η=0 不动 小步改善 过大会过冲(η=100 → 9.97)甚至改变哪些 ReLU 激活
⚠ 单步最优 ≠ 全程最优

停止： 定期看验证、用开发数据定停止规则、保留最佳验证检查点
⚠ 损失变化很小可能只是优化卡住，不是最优

M3 T3 第 2 课 • 混淆矩阵与四个指标

源页面：Accuracy, precision, recall + M3 T3 Metrics by hand ｜ 整理日期：2026-10-14 ｜ 对应考纲：Topic 3「混淆矩阵、accuracy、precision、recall、F1」

0. 第一件事：先定义哪一类是「正类」

> ⚠⚠ **TP、FP、FN、TN 的含义完全取决于这个选择。**
> 答题时**永远先声明哪一类算正类。**

本页的具体场景：**类别 1 = 「次品」**，类别 0 = 「非次品」。**预测为正 ⇒ 触发一次检验。**

	含义
FP（误报）	**一次不必要的检验**
FN（漏报）	**漏掉了一个次品**

> ⚠ **「正类」指的是被指定的那一类，不一定是「好的结果」。**

1. ⚠⚠⚠ 混淆矩阵的摆法——本课的约定（极易混，必记）

本课用「行 = 真实类别，列 = 预测类别」：

	预测为正	预测为负
真实为正	**TP**	**FN**
真实为负	**FP**	**TN**

⚠⚠ 三套不同的摆法，别搞混

来源	摆法
本课（考试按这个）	**行 = 真实，列 = 预测**，且**正类在前**
sklearn `confusion_matrix(y_true, y_pred, labels=[0,1])`	**类别 0 在前** ⇒ 输出是 **[[TN, FP], [FN, TP]]**
CS229 那份评估指标幻灯	列 = 真实（和本课**转置**）

> ★ 页面自己的提醒：**在给各个格子命名之前，先读清楚行标签和列标签。**

> ⚠ 考试时**先看题目给的是哪种摆法**，不要凭肌肉记忆填。

2. 四个核心指标

指标	公式	它回答的问题
Accuracy	(TP+TN)/(TP+TN+FP+FN)	**全部预测里有多大比例是对的?**
Precision	**TP/(TP+FP)**	**模型说是正的时候，有多少次是对的?**
Recall / TPR	**TP/(TP+FN)**	**所有真正的正例里，模型找到了多少?**
F1	**2PR/(P+R)**	**precision 与 recall 的调和平均**

> **⚠⚠ F1 公式里的 P 指 precision，不是预测概率。**
> R 指 recall。
> **TPR（真正率）是 recall 的另一个名字。**

各自什么时候重要

- **Accuracy**：**只有当类别频率和错误代价不会让「盲猜多数类」显得很强时**，它才最有信息量。
- **Precision**：**误报代价高**时重要。
- **Recall**：**漏报代价高**时重要。
- **⚠⚠ F1 忽略 TN，而且把 precision 和 recall 看得同等重要 ⇒ 它并不自动就是每个不平衡问题的正确指标。**

★ 为什么 F1 用调和平均而不是算术平均

$\text{precision} = 1,\ \text{recall} = 0.1$
算术平均 $= 0.55$
$F1 = 2(1)(0.1)/(1 + 0.1) \approx 0.182$ ← 被小的那个拖住

> **高 F1 要求两者都相当高。**
> **⚠ 但它既不考虑 TN，也没有说明金钱上的代价。**

3. ★★ 看分母来选指标

Precision 从「被预测为正」这一组出发： 分母 TP + FP
Recall 从「真实为正」这一组出发： 分母 TP + FN

> **★ 这两组有重叠，但不是同一组。**

页面的小例子：6 个案例被预测为正，但只有 5 个真的是正的，其中 4 个同时属于两组。

> Precision 的分母 = 6（预测为正的）
> Recall　的分母 = 5（真实为正的）

⚠ 计数版公式与零分母

> $F1 = 2TP / (2TP + FP + FN)$　　← 避免过早对 precision/recall 四舍五入

> **⚠ 若某个分母为零，这个比值在数学上无定义**
> **——要把这种情况报告出来，并说明你采用的约定。**
> **补充 practical 采用的约定是取 0。**

4. ★ 官方算例（12 个案例）

> 真实　[1,1,1,1,1, 0,0,0,0,0,0,0]
> 预测　[1,1,1,1,0, 1,1,0,0,0,0,0]

逐行比对：前四个 (1,1) ⇒ **4 个 TP**；第五个 (1,0) ⇒ **1 个 FN**；

第六、七个 (0,1) ⇒ **2 个 FP**；剩下五个 (0,0) ⇒ **5 个 TN**。

> **★ 计数必须加起来等于 12**
> —— 这是个很好的自查。

> TP=4　FN=1　FP=2　TN=5
>
> Accuracy $= (4+5)/12 = 9/12 = 0.75$
> Precision $= 4/(4+2) = 4/6 \approx 0.667$
> Recall $= 4/(4+1) = 4/5 = 0.8$
> F1 $= 2(0.667)(0.8)/(0.667+0.8) \approx 0.727$
> 或用计数版：$2(4)/(2\cdot4+2+1) = 8/11 \approx 0.727$ ✓

⚠ 自测：一个「把所有案例都预测为负」的分类器，在正例稀少的数据集上能有高准确率吗?

> **能。**
> 正例稀少时，**全判为负就能答对绝大多数**——准确率等于负例占比。
> ⇒ **这正是「accuracy 的基线是多数类占比」那条。**

5. ★★ 阈值与指标的关系

> **基于预测类别的指标都依赖判定阈值。**

在固定的分数上降低二分类阈值:

> 预测为正的数量　只增不减
> TP 和 FP　　都不会减少
> ⇒ Recall　　只要存在真实正例，就不会下降
> ⇒ Precision　　可能升，也可能降

> **⚠⚠ 在验证数据上、用应用场景的代价或约束来选阈值；不要在测试集上调它。**

6. ★★★ 从指标到决策：用代价把阈值定下来

设定：**漏掉一个次品代价 5 个单位，一次不必要的检验代价 1 个单位。**

> 总代价 = 5·FN + 1·FP

阈值	FN	FP	总代价 = 5FN + FP
0.5	6	4	**5(6)+4 = 34**
0.3	2	12	**5(2)+12 = 22** ← 更优

> **★ 在这组代价下，0.3 更好——尽管它产生了更多的误报。**

★★ 但把代价改成相等，结论就翻过来

> 代价相等时 总代价 = FN + FP
> 阈值 0.5：　6 + 4 = 10　← 这时 0.5 更优
> 阈值 0.3：　2 + 12 = 14

> **⚠⚠ 代价和产能是决策的一部分；不存在普适最优的阈值。**
> **⚠ 在用测试数据评估之前，把这个选择冻结。**

7. 配套活动：Metrics by hand（20 位顾客，广告点击）

Click=1 为正类。 ⚠ 只给了硬判定，所以这张表无法确定完整的 ROC 曲线或 AUC。

我把 20 行逐行比对的结果

类型	顾客编号	计数
TP (1,1)	1, 5, 7, 9, 14, 17	**6**
FN (1,0)	3, 12, 19	**3**
FP (0,1)	6, 16	**2**
TN (0,0)	2,4,8,10,11,13,15,18,20	**9**

> ✓ 自查：6+3+2+9 = **20**。真实正例 = TP+FN = **9**，真实负例 = FP+TN = **11**。

> $\text{Accuracy} = (6+9)/20 = 15/20 = 0.75$
> $\text{Precision} = 6/(6+2) = 6/8 = 0.75$
> $\text{Recall} = 6/(6+3) = 6/9 \approx 0.667$
> $\text{F1} = 2(6)/(2\cdot 6+2+3) = 12/17 \approx 0.706$

★ 和「永远预测不点击」的基线比

> 基线全判 0 ⇒ **在 11 个真实负例上全对** ⇒ **准确率 = 11/20 = 0.55**
>
> **模型的 0.75 确实高于基线 0.55。**

在广告场景里怎么解释这两个指标

- **Precision 0.75**：**模型说「会点击」的那 8 个人里，有 6 个真的点了。**

⇒ 直接关系到**投放预算的浪费程度。**

- **Recall 0.667**：**9 个真的会点击的人里，模型抓到了 6 个，漏掉 3 个。**

⇒ 直接关系到**错失的潜在收益。**

代价假设怎样影响取舍

> **展示一次广告的成本低、而一次点击的价值高**
> ⇒ **漏报（FN）更贵** ⇒ **应当偏向提高 recall**（降低阈值）。
>
> **反过来**，若展示成本高或会惹恼用户 ⇒ **误报（FP）更贵** ⇒ **偏向提高 precision。**

⚠ 还需要什么信息才能研究别的阈值 / 判断活动价值?

需要预测的分数或概率

（不只是硬判定）——**有了分数才能扫阈值、画 ROC、算 AUC。**

判断活动价值还需要**每次展示的成本和每次点击的收益。**

8. 总结自测

Q：一个分类器准确率更高，有没有可能对任务反而更没用?

完全可能。

★ 例如正例稀少时，**「全判为负」的准确率很高但毫无用处**（recall = 0，一个正例也找不到）。

准确率把两类错误一视同仁，而真实任务里它们的代价通常差很远。

⇒ **要看任务最不能容忍哪一类错误，再据此选指标。**

Q（页面的 Study check）：先构造混淆矩阵再套公式；然后说明应用最不能容忍哪种错误，并据此选指标。

这就是标准答题顺序：**①声明正类 → ②逐行比对得计数 → ③自查计数之和 → ④套公式 → ⑤结合代价解释。**

★ 速查卡

⚠ 先声明哪一类是正类!

本课摆法：行＝真实，列＝预测，正类在前

[TP FN]

[FP TN]

⚠ sklearn labels=[0,1] 给的是 [[TN,FP],[FN,TP]] —— 顺序不同!

Accuracy ＝ (TP+TN)/总数

Precision ＝ TP/(TP+FP)　分母＝「我预测为正的」

Recall ＝ TP/(TP+FN)　分母＝「真实为正的」，★ 固定不变

F1 ＝ 2PR/(P+R) = 2TP/(2TP+FP+FN)　⚠ 不含 TN

降阈值：预测为正只增不减 ⇒ recall 不降；precision 可升可降

选阈值：在验证集上用代价选，⚠ 不要在测试集上调

代价法：总代价 = c_FN·FN + c_FP·FP，★ 代价一变结论就可能翻转

M3 T3 第 3 课 • ROC 曲线与 AUC

源页面：ROC curves and AUC ｜ 整理日期：2026-10-14 ｜ 对应考纲：Topic 3「ROC/AUC 与阈值权衡」 ｜ ★ **M3 T3 Tutorial Q3 官方描述：「从分数得到 ROC 上的点、AUC 与阈值决策」**

0. 主旨

从**一个**混淆矩阵，走到**一串**阈值选择；然后**把「排序质量」和「决策质量」区分开。**

⚠ 全程保持模型和评估样本不变，只动阈值。

1. 先把定义摆清楚

TPR = TP/(TP+FN) = recall（召回率、真正率）
FPR = FP/(FP+TN) = 1 − specificity（假正率）
specificity = TN/(TN+FP)　真实负例里被正确拒绝的比例
prevalence = (TP+FN)/N　真实为正的案例占比

每个阈值产生一个点 (FPR, TPR)。

★ 曲线越靠近左上角，排序能力越强
——在错误接受很多负例之前，就已经找到很多正例。

⚠ **校准（calibration）问的是预测概率与观测事件频率是否一致；
光有好的排序（高 AUC）并不保证校准好。**

2. ★★ 怎么构造这条曲线（五步）

① 定义正类，并确保「分数越大 = 支持该类的证据越强」
（分数可以是概率，也可以是决策分数）
② 把真实正例数和真实负例数各数一次 —— ★ 这两个分母在整个过程中固定不变
③ 从比最大分数还高的阈值开始：全部预测为负 ⇒ 点 (0, 0)
④ 把阈值顺着「不同的分数值」往下降
⚠⚠ 共享同一个分数的观测必须一起加入
每次重新数 TP 和 FP，再分别除以那两个固定的总数
⑤ 以比最小分数还低的阈值结束：全部预测为正 ⇒ 点 (1, 1)
横轴画 FPR，纵轴画 TPR

⚠ **两个类别都必须存在**

，否则这两个率没有定义。

⚠⚠ **硬类别预测只能提供一个工作点——要画完整曲线必须用原始分数。**

⚠ **数据量小时，曲线是一串阶梯，不是一条光滑的物理定律。**

3. ★★ 官方算例：四个样本

A (0.90, 1)、B (0.80, 0)、C (0.40, 1)、D (0.10, 0)（格式为 分数, 标签）

2 个真实正例、2 个真实负例 ⇒ TPR = TP/2，FPR = FP/2 全程不变。

阈值	被预测为正	TPR	FPR	ROC 点
高于 0.90	无	0	0	**(0, 0)**
0.85	A	0.5	0	**(0, 0.5)**
0.60	A, B	0.5	0.5	**(0.5, 0.5)**
0.30	A, B, C	1	0.5	**(0.5, 1)**
0.05	A, B, C, D	1	1	**(1, 1)**

★★ 每加入一个正例，点向上移 1/2；每加入一个负例，点向右移 1/2。

这个「正例往上、负例往右」的规律是画 ROC 的核心直觉。

自测：阈值从 0.60 降到 0.30，哪个样本进来？点往哪移？

C 进来（分数 0.40）。C 是正例 ⇒ 点向上移

，从 (0.5, 0.5) 到 **(0.5, 1)**。

4. ★★ 梯形法算 AUC

$$\mathrm{AUC} \approx \Sigma_i \, (\mathrm{FPR}_{i+1} - \mathrm{FPR}_i) \times (\mathrm{TPR}_i + \mathrm{TPR}_{i+1})/2$$

⚠ **FPR = 0 处那段竖直线段贡献的面积为零**

（横向宽度为 0）。

本例计算：

$$\begin{aligned}(0.5-0) \times (0.5+0.5)/2 &= 0.5 \times 0.5 = 0.25\\ (1-0.5) \times (1+1)/2 \quad &= 0.5 \times 1 \quad = 0.50\\ \mathrm{AUC} &= 0.75\end{aligned}$$

⚠ **要把两个端点和每一段都算进去；竖直段横宽为零，所以面积为零。**

★ **用全部不同阈值画出的分段线性经验 ROC 曲线，梯形法给出的就是它的精确面积。**

若只给了少数几个工作点，结果是基于直线插值的估计。

5. ★★★ 用「正负配对」复核面积（更好用的办法）

AUC 的排序解释

随机取一个正例、随机取一个负例，正例分数更高的概率（平局另算）。

本例：正例 **A=0.90, C=0.40**；负例 **B=0.80, D=0.10**。

四个「正–负」配对：

配对	比较	结果
A vs B	0.90 > 0.80	**正例赢** ✓
A vs D	0.90 > 0.10	**正例赢** ✓
C vs B	0.40 < 0.80	**负例赢** ✗
C vs D	0.40 > 0.10	**正例赢** ✓

3 胜 / 4 对 ⇒ AUC = 0.75　　✓ 和梯形法完全一致

★ **一个平局算半胜。**

★★ **这个办法不需要选任何阈值**，考场上比画梯形快得多，**而且能互相验算。**

6. ⚠ AUC 数值的解读分寸（措辞很讲究）

情况	正确的说法
AUC = 1	**在被评估的这个样本上排序完美**，⚠ **不保证未来的准确率**
AUC = 0.5	**平均配对顺序处于随机水平**；⚠ **不等于「在每个阈值上都没有有用的行为」**
AUC < 0.5	⚠⚠ **应当去检查分数方向、标签约定和抽样，而不是拿测试结果去自动把模型反过来用**

⚠ AUC 接近 0.5 时，**单条经验曲线也不必恰好落在对角线上。**

7. ★★★ AUC 不能决定什么（四条，最可能出选择题）

> 1. **它不替你选工作阈值。**
> 2. **它不衡量概率校准。**
> 3. **它不编码误报和漏报的真实代价。**
> 4. **负例很多时，它可能掩盖掉数量不可接受的误报。**

★ 第 4 条的机理（页面说得很准）：

> **ROC 用的是「类内条件率」**
> ，所以**单独改变 prevalence 不一定改变 ROC AUC**，
> **但 precision 和实际可用性可能变化很大。**
> ⇒ **还要同时看 precision–recall 和 prevalence。**

> ⚠ **用验证数据加上应用场景的代价、产能或安全约束来选最终阈值。在没碰过的测试集上评估一次。**

自测：两个系统 AUC 相同，它们的可用阈值或误报代价一定相同吗?

> **不一定。**
> **AUC 概括的是整条曲线（排序能力），而不是某一个工作点。**
> 两条形状不同的曲线**完全可以有相同的面积**，但在你实际会用的那个 FPR 附近表现差很远。

8. ★★ 同一组样本的 PR 视角（对照着看）

PR 曲线：横轴 recall，纵轴 precision。 还是 A/B/C/D 那四个：

阈值	TP	FP	FN	Recall	Precision
0.85	1	0	1	0.5	**1**
0.60	1	1	1	0.5	**0.5**
0.30	2	1	0	1	**2/3**
0.05	2	2	0	1	**0.5**

> ⚠ **高于 0.90 时没有任何正类预测 ⇒ precision 的分母为零**；绘图库可能自行提供一个端点约定。

> **★★ 注意 precision 这一列：0.5 → 2/3 → 0.5，先升后降 —— 阈值下降时 precision 不必单调变化。**
> **随机排序的无技能参考线 = 正例占比 = 2/4 = 0.5。**

9. ★★★ 稀有正例的那个算例（本页最该记的数字）

1000 个观测，其中只有 10 个正例（⇒ 990 个负例）。

在 TPR = 0.8、FPR = 0.1 处：

$$\text{TP} = 0.8 \times 10 = 8 \qquad \text{FN} = 10 - 8 = 2$$
$$\text{FP} = 0.1 \times 990 = 99$$

$$\text{Precision} = 8/(8 + 99) = 8/107 \approx 7.5\%$$
$$\text{Recall} \quad = 8/10 \quad = 80\%$$

★★ recall 高达 80%，precision 却只有约 7.5%。

原因：FPR 虽然只有 0.1，但底下有 990 个负例，于是产生了 99 次误报。

★ 页面的总结很精辟：

ROC 衡量的是「类内条件的排序行为」；而 precision 揭示的是「在这个 prevalence 下，正类预测的实际质量」。

⚠ 正例稀少时两种视角都要看

，并用验证数据和错误代价来选工作阈值。

⚠ 一条范围说明

不要机械地把 ROC 的梯形配方套用到「平均精度（average precision）」上——

average precision 和梯形法算的 PR 面积是两个不同的概括量。

本 Topic 不要求计算这些概括量。

10. 总结自测

Q：80% 的 recall 怎么能和约 7.5% 的 precision 并存?

因为**两者的分母完全不同**：

recall 的分母是 10 个真实正例（抓到 8 个，很好看）；

precision 的分母是 107 个「被预测为正」的案例（其中 99 个是误报）。

⇒ **负例基数巨大时，很小的 FPR 也会压倒性地稀释 precision。**

Q（页面的 Study check）：给一张「分数 + 标签」的小表，怎么做?

**① 按分数排序 → ② 让阈值依次穿过每个不同的分数值（⚠ 相同分数的观测要一起处理）

→ ③ 算出每个 ROC 点 → ④ 用梯形估计 AUC。**

★ 算完用「正负配对法」复核一遍

，两种方法应当一致。

★ 速查卡

$TPR = TP/(TP+FN) = recall$　　$FPR = FP/(FP+TN) = 1 - specificity$

画 ROC：分母(真实正例数、真实负例数)固定不变
从 (0,0) 开始，阈值往下降，⚠ 同分数的样本一起加
加正例 → 往上走　加负例 → 往右走
到 (1,1) 结束

AUC 两种算法：
梯形：$\Sigma\ (\Delta FPR) \times (TPR_i + TPR_{i+1})/2$　⚠ 竖直段面积为 0
配对：P[随机正例 > 随机负例]，★ 平局算半胜 —— 更快，且能互验

⚠ AUC 不决定：阈值 / 校准 / 真实代价；负例多时会掩盖误报数量

★ 稀有正例算例：1000 个里 10 个正例，TPR=0.8 FPR=0.1
⇒ TP=8, FN=2, FP=99 ⇒ recall 80% 但 precision 仅 7.5%